BTS4410 – Obligatorisk gruppeoppgave 3

Oppgaven skal løses i grupper bestående av maks. 1-3 studenter.

Part A: The Homegrown Crypto Bar

Baren «The Homegrown Crypto Bar» utsteder drikkebonger i digital form: «HCB_PREPAID_BEER.BIN».

Disse drikkebongene er kryptert med AES-CTR, og de har et opplegg der en i kassa dekrypterer HCB_PREPAID_BEER.BIN. Dermed ser hvor mange øl bongen er gyldig for. Når bongen er brukt, blir IV notert som brukt. Dermed kan ikke bongen brukes mer enn en gang.

Selve fila HCB_PREPAID_BEER.BIN er kodet som følgende:

```
IV/CTR    : 16 bytes
Ciphertext: 66 bytes
```

Dere har vært så heldige å få tak i en plaintext, og denne er gitt i utf-8 kodet tekst.
Siden dere vet at CTR er en stream-cipher mode, så ønsker dere å se om dere kan utnytte dette.

Plaintext: `*** The Homegrown Crypto Bar --- Beer voucher. Value: 1 beers. ***`

I Python blir dette representert slik:

```
message = b"*** The Homegrown Crypto Bar --- Beer voucher. Value: 1 beers. ***"
```

Vi ser at antallet øl er kodet inn som et siffer, dvs. sifferet er kodet som tekst.

Dere ønsker å gjøre å gjøre et forsøk der dere kjøper en drikkebong verdt **1** øl. Dere har som hypotese at baren *ikke* har noen form for integritetsbeskyttelse på bongen, og det derfor burde være mulig å modifisere antallet øl (dvs: vi antar at ciphertext er «mallable»).

Oppgave:

- Anta at dere har kjøpt en drikkebong på **1** øl («HCB_PREPAID_BEER.BIN»).
- Dere vet dermed innholdet i ciphertext'en.
- Dere kjenner strukturen på fila og dere kjenner både til plaintext og formatet.
- Modifisert antallet fra **1** til et annet tall – for beskjedenhets skyld: **2**
- Lag Python kode som gjør dette
    - Lag en XOR maske som endrer «1» til «2»
    - Finn posisjonen til antallet øl (gjør dette i plaintext) og regn ut posisjonen i HCB_PREPAID_BEER.BIN
    - Les inn HCB_PREPAID_BEER.BIN
    - Utfør XOR (med maske) på riktig posisjon i HCB_PREPAID_BEER.BIN data
    - Lagre HCB_PREPAID_BEER.BIN

- Python kode og den modifiserte HCB_PREPAID_BEER.BIN fila skal leveres.
- Dokumentasjon, beskrivelse og uttestings data skal leveres.

BTS4410 – Obligatorisk gruppeoppgave 3

Part B: ML-SUCI

Det skal utvikles en SUCI variant som er basert på PQC.
Helt konkret er denne SUCI varianten basert på ML-KEM 768.

I likhet med SUCI oppgaven så skal ML-SUCI ha KEYGEN, CONCEAL og DECONCEAL funksjonalitet.

ML_Home.py:

- KEYGEN: Genererer ML-KEM 768 nøkkelpar som lagres i PEM-filer
- DECONCEAL: Bruker *decapsulation* nøkkel til å hente fra delt nøkkel, som så brukes til å dekryptere selve innholdet.

ML_User.py:

- Får tilgang på ML-KEM 768 *encapsulation* nøkkel (i en PEM-fil).
- CONCEAL: Bruker encapsulation-prosessen til å lege en delt nøkkel/ciphertext. Nøkkelen brukes til å kryptere selve innholdet.

Filformatet for **ML-SUCI.BIN** er litt annerledes enn for **SUCI.BIN**.

```
IV:                12 bytes
Home Identifier:   <2 bytes length prefix><actual length>
ML-KEM ciphertext: <2 bytes length prefix><actual length>
Encrypted user Id: <128 bytes><16-byte tag>
```

Tips: **ML_User.py CONCEAL** printer ut de faktiske lengdene.

Oppgave:

Oppgaven består i å skrive **DECONCEAL** delen av **ML_Home.py** .

Dere skal dokumentere at dette er testet, både mot vedlagte **ML_SUCI_data.bin** og mot en annen gruppe (med annen implementasjon).

BTS4410 – Obligatorisk gruppeoppgave 3

Part C: Enkel hastighetstest: BLAKE3 vs SHA3

Blake3 finnes på https://pypi.org/project/blake3/

Oppgaven består i å gjøre en enkel test av hastigheten for Blake3[1] vs SHA3. Her kan dere benytte timing verktøy i OS eller i python (time modulen har noe slik funksjonalitet[2]).

Testen består av 3 deler:

a) Liten datablob - datablob1.bytes
b) Medium datablob - datablob2.bytes
c) Stor datablob - datablob3.bytes

Testene bør kjøres minst 100.000 ganger hver, for både Blake3 og SHA3.
For **datablob3.bytes** kan dere kjøre testen 10.000 ganger.

Dere bør sørge for at tiden som går med til fil-lesning IKKE teller med i timingen.

Oppgave:

Lag en fin tabell for testene. Tips: Ikke over-tolk resultatene.

Part D: Mini-test av Ascon

Ascon finnes på pypi, men det er bedre å bruke den versjonen som finnes på https://github.com/meichlseder/pyascon (denne er lagd av designerne av Ascon).

Oppgaven er enkel:

a) Lag et python program som sjekker Ascon hash tags for filer.

Vedlagt finnes 5 eksempler på tekstfiler med tag («.asig»).

Signaturene er beregnet med Ascon-Hash256, og de er skrevet ut i Base64.

```
Hash-algo : Ascon-Hash256
Datetime  : 2026-09-29 10:15:43
Filename  : test1.txt
Base64-sig: yvcBR0VLLqswQWKBOnlelifOPasteRb4Y6gmJT0Ywno=
```

Eksempel: Dette er **test1.txt.asig** fila

Oppgaven går ut på å sjekke om de vedlagte test-filene faktisk matcher sine respektive «.asig» sjekksummer. Dokumenter resultatet og legg ved python filer, med mer.

---

[1] Blake3 har egen funksjonalitet for «multi threading» og «memory mapped IO», og dette kan gi store utslag for store filer og store objekter. Dette kan dere bruke, men dette kommer da i tillegg til de «ordinære» testene.
[2] Se f.eks. time.perf_counter() og/eller time.perf_counter_ns().